

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	MAGESH.S	Reg. No.:	192525002
Section / Batch:	B. Tech AIML	Date:	14/07/2026

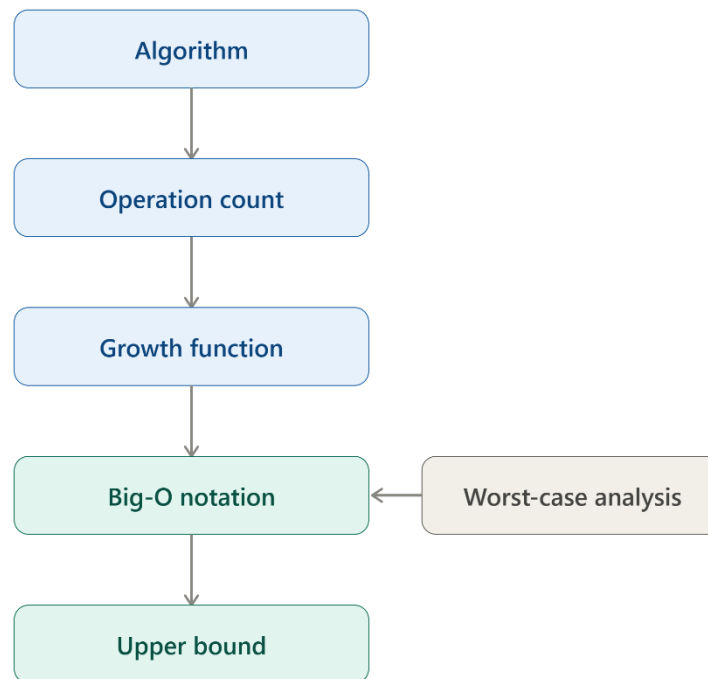
Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONCEPT MAPPING

Q 1. Consider the concept map: Algorithm → Operation Count → Growth Function → Big-O → Upper Bound Map how counting operations leads to asymptotic upper bounds. Extend the map with Worst-case Analysis. Justify why Big-O represents worst-case behavior. Interpret how this supports algorithm comparison.

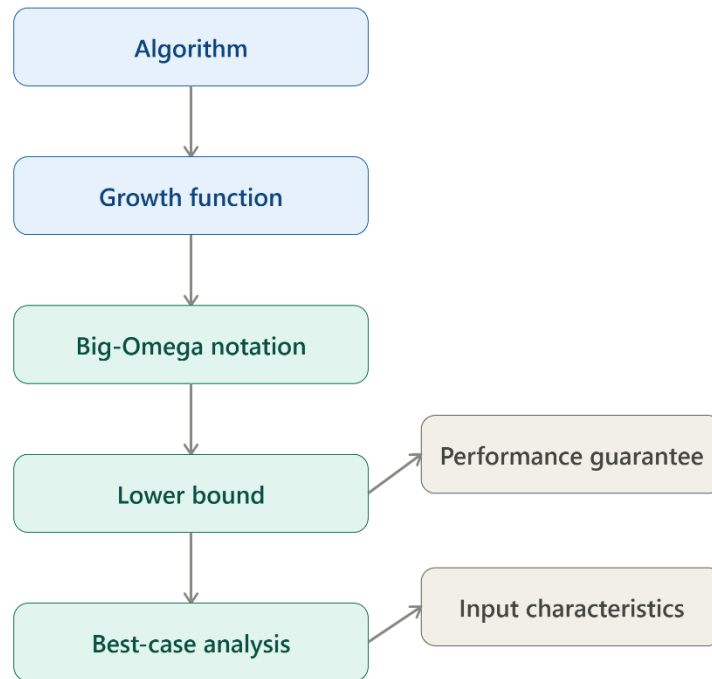


Explanation: Analyzing an Algorithm starts with counting its basic Operations (comparisons, assignments, arithmetic steps) as a function of input size. This operation count is generalized into a Growth Function — an expression describing how the count increases as n grows (e.g., n , n^2 , $\log n$).
Extended link — Worst-case Analysis: Big-O is connected to Worst-case Analysis because Big-O specifically captures the *maximum* possible operation count for any input of size n , which is exactly what worst-case analysis measures.

Justification: Big-O represents an upper limit rather than an exact value because real-world algorithm behavior varies with different inputs of the same size (e.g., a nearly-sorted array vs. a reverse-sorted array for Insertion Sort). By focusing on the worst-case ceiling, Big-O gives a safety guarantee — the algorithm will never perform worse than this bound, regardless of input variation.

Interpretation: This map shows a funnel from concrete operation counting to abstract classification, ending in a guarantee. This upper-bound guarantee is critical for reliable performance estimation, especially in systems where predictable worst-case behavior matters (e.g., real-time systems, safety-critical software).

Q2. Consider the concept map: Algorithm → Growth Function → Big-Omega → Lower Bound → Best-case Explain how lower bound represents minimum execution time. Extend the map with performance guarantee node. Justify why best-case analysis is useful. Interpret how this contributes to overall performance understanding.

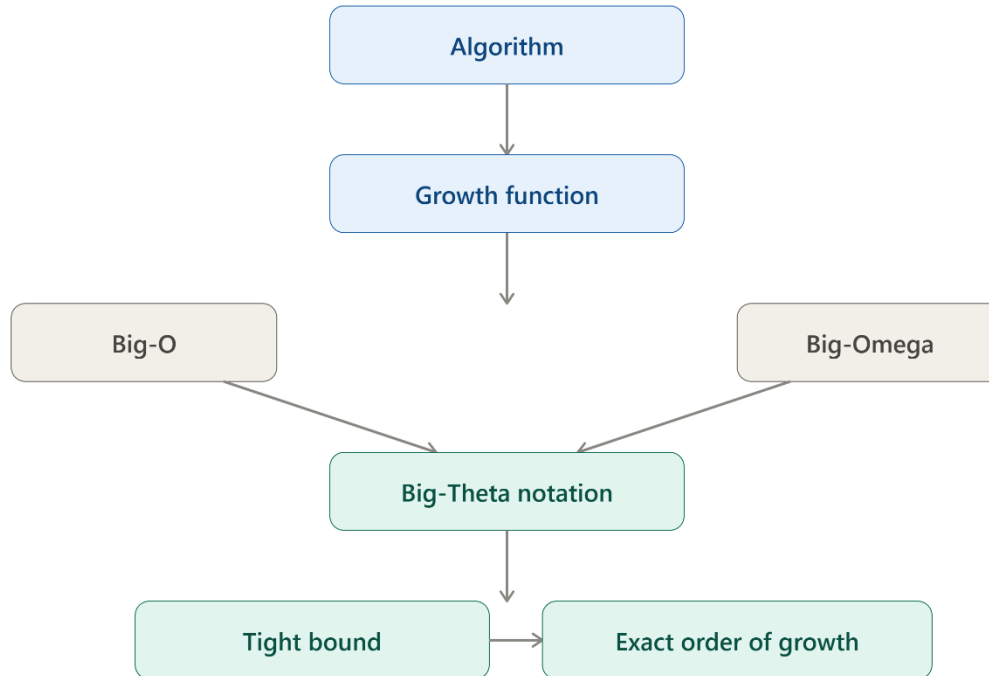


Explanation: Just as Big-O captures maximum growth, Big-Omega (Ω) captures the Lower Bound — the minimum amount of time/operations an algorithm will *always* require, no matter how favorable the input. This lower bound directly corresponds to Best-case Analysis, since it represents the fastest scenario possible.

Justification: Big-Omega is useful because it establishes a guaranteed minimum efficiency — this matters in scenarios where we want to know if an algorithm has any hope of being fast, or to prove that no algorithm can do better than a certain bound (a lower-bound proof for a whole problem class, not just one algorithm).

Interpretation: While Big-O warns us about worst-case risk, Big-Omega tells us about best-case potential. Together, they give a balanced view — neither overly pessimistic nor overly optimistic — of how an algorithm can behave across different input conditions.

Q3. Consider the concept map: Algorithm → Growth Function → Big-Theta → Tight Bound → Exact Complexity Explain how tight bound represents exact growth. Extend the map by linking Big-O and Big-Omega. Justify why Big-Theta is preferred for precise analysis. Interpret how this improves decision making.



Explanation: Big-Theta (Θ) represents a Tight Bound — it applies only when the upper bound (Big-O) and lower bound (Big-Omega) are the *same* growth rate. When this happens, we know the algorithm's growth is neither faster nor slower than this rate; it is exactly this rate asymptotically.

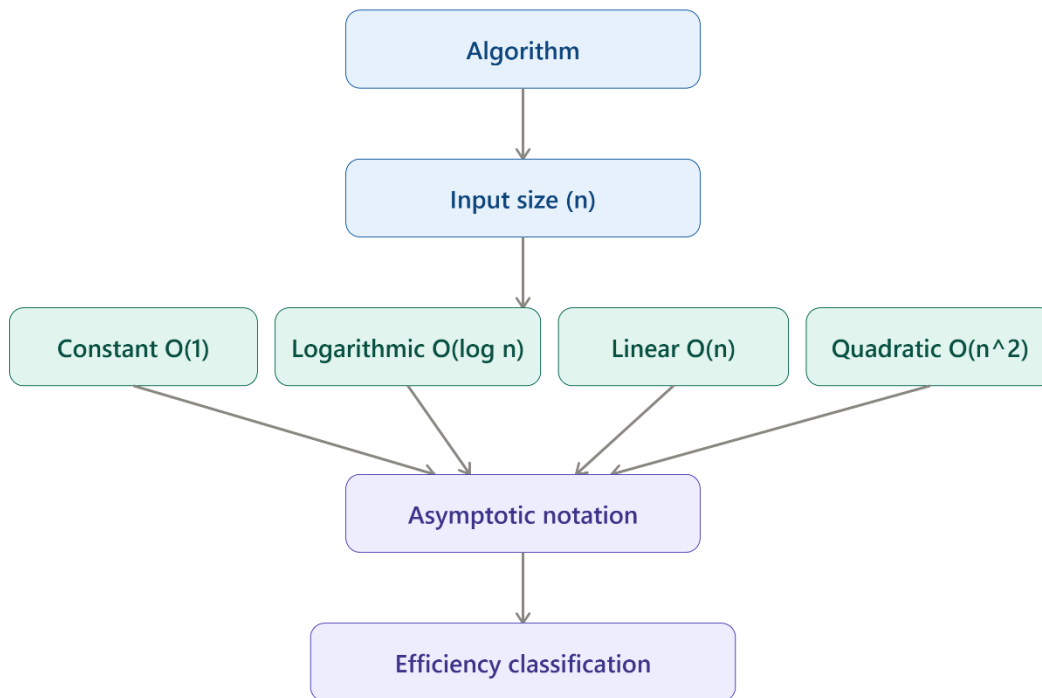
Extended links — Big-O and Big-Omega feeding into Big-Theta: Θ is formally defined as the intersection of O and Ω : $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$. This shows the two bounds converging into one precise classification.

Justification: Big-Theta is preferred when the exact asymptotic growth is known because it gives the most informative and precise description — no ambiguity about whether the actual bound is tighter or looser. For example, saying Merge Sort is $\Theta(n \log n)$ is stronger and more useful than just saying it's $O(n \log n)$, since Θ confirms this is both the best and worst case growth rate.

Interpretation: This map distinguishes between *approximate* bounds (O alone, or Ω alone — which only bound from one side) and a *precise* description (Θ , which pins down the exact behavior). This distinction matters when comparing algorithms rigorously, since Θ gives an unambiguous, single classification

Q4. Consider the concept map: Algorithm → Input Size → Growth Rate → Complexity Classes

Explain how algorithms are classified based on growth. Extend the map by adding constant, linear, logarithmic, and quadratic classes. Justify how classification helps in comparison. Interpret how the map supports scalability analysis.



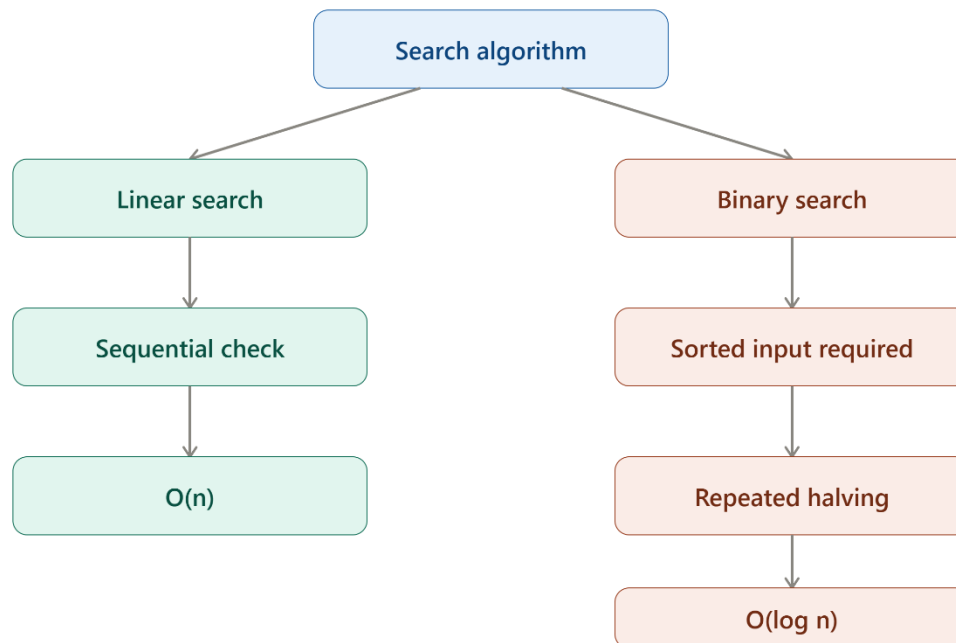
Explanation: As Input Size increases, different algorithms exhibit different Growth Patterns:

- Constant Time $O(1)$: operations don't depend on n (e.g., accessing an array element by index).
- Logarithmic Time $O(\log n)$: input is repeatedly halved (e.g., Binary Search).
- Linear Time $O(n)$: operations grow proportionally with n (e.g., a single loop through an array).
- Quadratic Time $O(n^2)$: operations grow with the square of n (e.g., nested loops, Bubble Sort).
-

Justification: Logarithmic and linear algorithms are generally more scalable than quadratic algorithms because their operation count grows much more slowly as n increases. For example, doubling the input size roughly doubles the work for a linear algorithm.

Interpretation: This map provides a practical taxonomy for classifying algorithms — from best (constant) to worst (quadratic) among these four — helping developers immediately gauge how an algorithm will scale before even running it, purely from its complexity class.

Q5 Consider the concept map: Search Algorithms → Linear Search → $O(n)$ and Binary Search → $O(\log n)$ Map the relationship between search techniques and their complexities. Extend the map with input condition node for binary search. Justify why binary search is more efficient. Interpret how this map supports algorithm selection.



Explanation: Both Linear Search and Binary Search are strategies under the broader category of Search Algorithms, but they take fundamentally different approaches:

- Linear Search checks each element sequentially until a match is found (or the list ends), resulting in $O(n)$ complexity in the worst case.
- Binary Search repeatedly halves the search space by comparing the target to the middle element, resulting in $O(\log n)$ complexity — but this requires the crucial Input Requirement that the data be sorted beforehand.

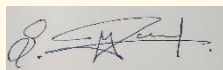
Justification for Binary Search's superior growth rate: Because Binary Search eliminates half of the remaining elements with each comparison, the number of steps needed grows logarithmically rather than linearly. For a list of 1,000,000 elements, Linear Search may need up to 1,000,000 comparisons, while Binary Search needs at most ~ 20 (since $\log_2(1,000,000) \approx 20$) — a massive efficiency gain for large inputs.

Interpretation: This comparative map highlights a key trade-off: Binary Search offers dramatically better performance, but only when preprocessing (sorting) is feasible and the input requirement is satisfied. If data is unsorted and sorting cost outweighs benefits (e.g., a single search on an unsorted list), Linear Search may still be the practical choice — showing how the map supports informed algorithm selection based on both efficiency and constraints.

Code of Conduct

I MAGESH.S (192525002) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:


RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

